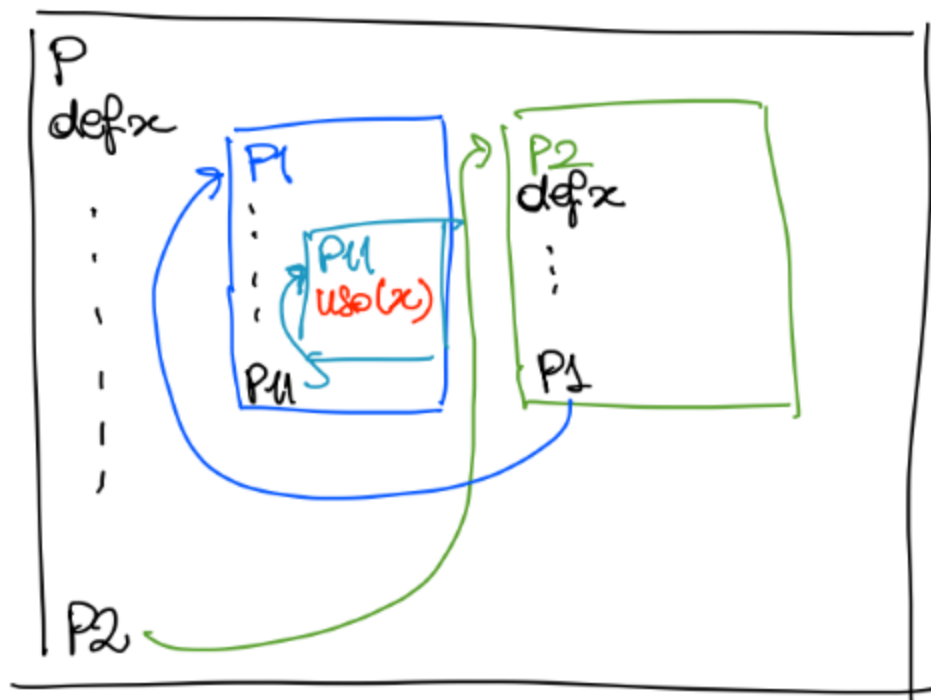


Regole di scoping → servono per risolvere riferimenti non locali presenti in procedure ovvero dove il contesto di definizione può non coincidere con il contesto di esecuzione

↳ **statico**
l'ambiente di riferimento è quello della def.

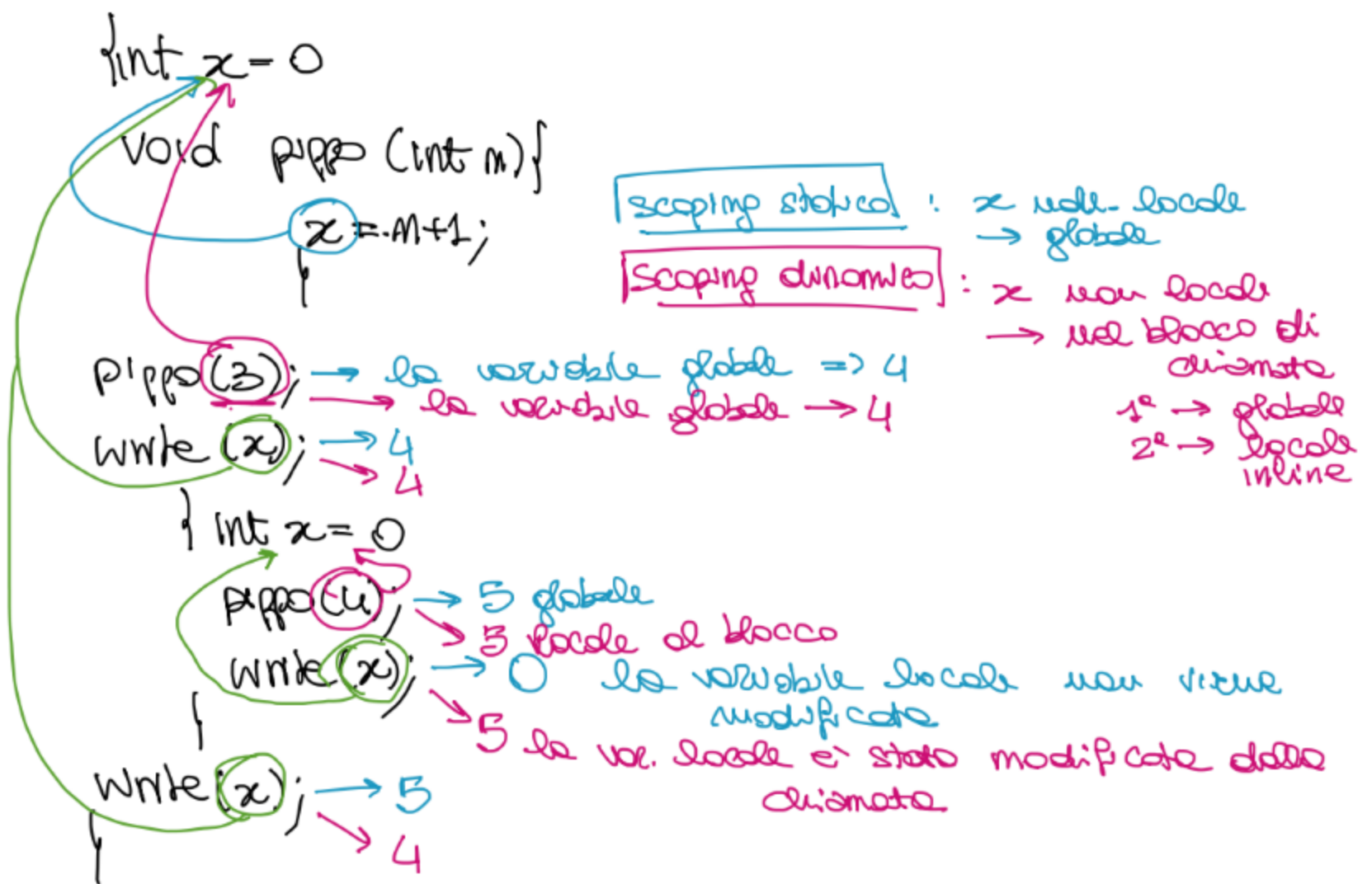
↳ **dinamico**
l'ambiente di riferimento è quello dell'esecuzione



Scoping statico: $P \begin{cases} P1 \\ P2 \end{cases}$

$P \xrightarrow{\text{def } x} P1 \xrightarrow{\text{uso}(x)} P11$
 $P \xrightarrow{\text{def } x} P2 \xrightarrow{\text{def } x}$

Scoping dinamico: $P \xrightarrow{\text{def } x} P2 \xrightarrow{\text{def } x} P1 \xrightarrow{\text{uso}(x)} P11$



I risultati possono essere diversi se:

- Il codice contiene una procedura con ambiente non locale
- Esistono diverse definizioni, di questi riferimenti non locali, nel codice

ESERCIZI

↳ Scoping statico - dinamico

① `int i;`

codice
incompleto

⊗

`for (int i=0; i <= 1; i++) {`

`int x;`

`x = fun();`

Ambiente della chiamata
↳ `i, x`

`}`

Fornire il codice da inserire in ⊗ in modo
tale che: è commentare spiegando perché scrivere esattamente il codice che scriverete

1) Se il PL usa scoping statico allora
le due iterazioni del for assegnano a `x`
lo stesso valore

2) Se il PL usa scoping dinamico allora
le due iterazioni devono dare risultati
diversi

Mostare poi le due esecuzioni

Cosa dobbiamo fare:

↳ renderlo eseguibile: dobbiamo definire il
codice di `fun()`

↳ fare in modo che le regole di scoping
siano le seguenti: ① dobbiamo
avere una procedura (`fun`) con
ambiente non locale

② i riferimenti non locali di `fun`
devono essere definiti sia nel
contesto di `def` che di chiamata di `fun`

Def: `int fun ()` { le uniche variabili
non locali utilizzabili
sono `x` e `i` }

⇒ qualunque altra variabile
potrebbe essere inserita
suo nell'ambiente globale
non potrebbe differenziare la
regola di scoping

`int fun ()` { ~~`x=3`~~ / ~~`i=2`~~ renderebbe il valore
indipendente dal riferimento

⇒ La scelta migliore è `i` che è già
inizializzato nell'ambiente di chiamata che
non è modificabile ⇒ `x` non può essere
inizializzato dentro il `for`.

Nel codice Ⓢ dobbiamo inizializzare `i` e definire
`fun` in modo che utilizzi `i` per calcolare il
risultato

Ⓢ : `i = 1;`
`int fun ()` { `return (i);` }

`int i;`
`i = 1;`
`int fun ()` { `return (i);` }
`for (int i = 0; i <= 1; i++)`
 `int x;`
 `x = fun();`

scoping statico

scoping dinamico

Scoping statico → Le due iterazioni non modificano
la ~~il~~ globale che quindi viene
restituita = 1 in entrambe le iterazioni

Scoping dinamico → Nelle due iterazioni il valore
prima 0 e poi 1 quindi la
chiamata restituisce rispettivamente
i due valori nelle due iterazioni

② { int a=0; int y=1;
⊛
void exec() { int x;
⊛⊛
y = fun();
a = a+1;
}
while (a <= 1) { exec(); }

le due esecuzioni
di fun()
↙ ↘
≠ nel =
dinamico nello
 statico

③ { int a=0;
⊛
while (a <= 1) {
 int x;
 ⊛⊛
 x = fun();
 a++;
}

come sopra

INDUZIONE STRUTTURALE

Def. BTree : $\begin{cases} \text{foglio} \in \text{BTree} & \text{base} \\ T_1, T_2 \in \text{BTree} \Rightarrow \text{branch}(T_1, T_2) \in \text{BTree} & \text{passo induttivo} \end{cases}$

Def $m: \text{BTree} \rightarrow \mathbb{N}$ conta il n° di nodi

$$\begin{cases} m(\text{foglio}) = 1 \\ m(\text{branch}(T_1, T_2)) = 1 + m(T_1) + m(T_2) \end{cases}$$

Def $h: \text{BTree} \rightarrow \mathbb{N}$ calcola l'altezza

$$\begin{cases} h(\text{foglio}) = 0 \\ h(\text{branch}(T_1, T_2)) = 1 + \max(h(T_1), h(T_2)) \end{cases}$$

→ prende il valore più grande tra i due



Dimostrare che $m(T) \leq \underline{2^{h(T)+1} - 1}$

Induzione strutturale:

BASE $T = \text{foglio}$ $m(T) = 1$ $h(T) = 0$

$$2^{h(T)+1} - 1 = 2^{0+1} - 1 = 2^1 - 1 = 1 = m(T)$$

se = allora $\leq$

PASSO INDUTTIVO $T = \text{branch}(T_1, T_2)$

Ip. induttiva la proprietà vale su T_1 e T_2

$$m(T_1) \leq 2^{h(T_1)+1} - 1$$

$$m(T_2) \leq 2^{h(T_2)+1} - 1$$

tesi la proprietà vale per T

$$m(T) \leq 2^{h(T)+1} - 1$$

L

$$m(T) = 1 + m(T_1) + m(T_2)$$

$$h(T) = 1 + \max(h(T_1), h(T_2)) \stackrel{\text{def}}{=} t$$

$$m(T) = 1 + m(T_1) + m(T_2) \stackrel{\text{H.p. ind.}}{\leq} 1 + (2^{h(T_1)+1} - 1) + (2^{h(T_2)+1} - 1)$$

$$\begin{cases} t = 1 + \max(h(T_1), h(T_2)) > \max(h(T_1), h(T_2)) \geq h(T_1) \\ t > h(T_1) \Rightarrow t \geq h(T_1) + 1 \\ t > h(T_2) \Rightarrow t \geq h(T_2) + 1 \end{cases}$$

$$5 > 4$$

$$5 > 2$$

$$5 \geq 4+1$$

$$5 \geq 3$$

$$m(T) \leq 1 + (2^{h(T_1)+1} - 1) + (2^{h(T_2)+1} - 1)$$

$$\leq 1 + (2^t - 1) + (2^t - 1) = \cancel{1} + 2^t - \cancel{1} + 2^t - 1$$

$$= 2 \cdot 2^t - 1 = 2^{t+1} - 1$$

$$= 2^{h(T)} - 1 \quad \checkmark$$

2

$$\begin{cases} e(\text{leaf}) = 1 \\ e(\text{branch}(T_1, T_2)) = e(T_1) + e(T_2) \end{cases}$$

$$\text{Dim } e(T) \leq 2^{h(T)}$$